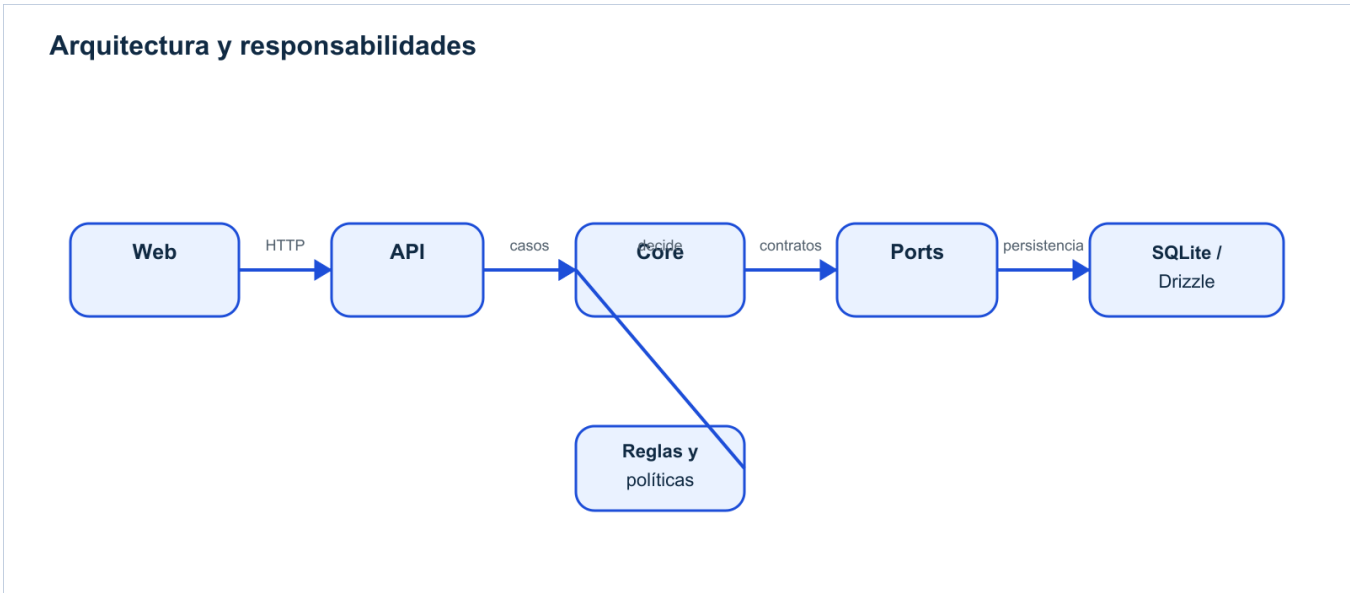
OpsKit | Arquitectura y portabilidad

Estructura para preservar reglas de negocio, aislar infraestructura y planear un cambio de almacenamiento sin falsos supuestos.

Dirección Web → API → Core → Ports → SQLite/Drizzle.	**Autoridad** El core decide; la base de datos protege invariantes físicos.
Beneficio Cambiar almacenamiento no debe reescribir las reglas operativas.	**Límite** NoSQL no es un reemplazo mecánico: exige modelo y consultas propios.

Arquitectura en capas



La Web presenta la operación; la API expone la entrada; el Core aplica casos de uso y políticas; los Ports definen contratos; SQLite/Drizzle implementa persistencia.

Web Navegación, formularios y estados visibles. No decide políticas de dominio.	**API** Valida el borde y dirige solicitudes a capacidades del Core.
Core Decide reglas: estados, transiciones, permisos y cierres.	**Ports** Aíslan contratos de repositorios, transacciones y dependencias.
SQLite / Drizzle Persiste datos, aplica restricciones y soporta consultas.	**Settings** Configura políticas sin convertir la interfaz o la BD en fuente de verdad.

Quién decide y quién protege

Capa	Responsabilidad	Ejemplo
Core	Decisión de negocio	Solo calculated puede cerrar un período.
Port	Contrato intercambiable	Repositorio de facturas o caja.
SQLite / Drizzle	Persistencia y protección	Restricciones, integridad y unidad de trabajo.
Web / API	Experiencia y borde	Presentar error y transportar la solicitud.

Regla de diseño

La base de datos no reemplaza los casos de uso. Sus restricciones son un respaldo físico; el Core conserva la decisión y el orden de las transiciones.

Portabilidad real

No copiar SQL Un motor nuevo requiere un adaptador que implemente los Ports, no SQL trasladado a ciegas.	**No omitir migraciones** El cambio debe incluir migración, validación de datos y plan de reversa.
No ignorar concurrencia Aislamiento, bloqueos, transacciones e idempotencia cambian por motor.	**NoSQL con modelo propio** Documentos, claves y patrones de consulta se diseñan según acceso, no como tablas disfrazadas.

Cambio de base de datos: plan mínimo

Frente	Acción concreta	Riesgo que reduce
Contrato	Implementar el adaptador de Ports	Dependencia del motor en el Core.
Datos	Diseñar y ejecutar migraciones verificables	Pérdida, tipos inconsistentes o saldos erróneos.
Concurrencia	Revisar UoW, transacciones, bloqueo e idempotencia	Doble cobro, cierre simultáneo o carreras.
Consultas	Medir índices y patrones de lectura/escritura	Regresión de rendimiento.
Operación	Respaldos, observabilidad y reversa	Recuperación incompleta ante fallo.

Criterio de salida

El cambio es aceptable cuando conserva las reglas del Core, pasa migraciones y pruebas de concurrencia, y demuestra que los procesos de caja, facturación, inventario y compensación mantienen su trazabilidad.